Mise en place d'un écosystème d'IA

420-A57-BB

TRAVAIL PRATIQUE

Module 3 : Environnement de développement et de déploiement IA

100% Google Colab

Professeur : Dr Komi SODOKE

Environnement	Google Colab (navigateur + compte Google)
Prérequis	Module 2 terminé · Compte Google actif

Préparation : accéder à Google Colab

Étape 1 : Ouvrez votre navigateur et allez sur colab.research.google.com

Étape 2 : Connectez-vous avec votre compte Google.

Étape 3 : Créez un nouveau notebook : Fichier → Nouveau notebook.

Étape 4 : Activez le GPU : Exécution → Modifier le type d'exécution → GPU → T4 → Enregistrer.

Convention : chaque exercice correspond à une ou plusieurs cellules Colab. Les blocs jaune clair indiquent le code à taper dans une cellule.

Partie 1 — Python et son écosystème

Exercice 1.1 — Explorer l'environnement Colab

Exécutez les cellules suivantes pour découvrir l'environnement Colab :

Cellule Colab

```
import sys
import platform

print(f'Python : {sys.version}')
print(f'OS : {platform.platform()}')
print(f'Machine : {platform.machine()}')

# Vérifier le GPU
!nvidia-smi
```

Question : Quel système d'exploitation utilise Colab ? Quelle version de Python ? Quel GPU est attribué à votre session ?

Exercice 1.2 — Bibliothèques pré-installées

Vérifiez quelles bibliothèques sont déjà disponibles :

Cellule Colab

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import torch
import tensorflow as tf

print(f'NumPy : {np.__version__}')
print(f'Pandas : {pd.__version__}')
print(f'Scikit-learn : {sklearn.__version__}')
print(f'PyTorch : {torch.__version__}')
print(f'TensorFlow : {tf.__version__}')
print(f'CUDA dispo : {torch.cuda.is_available()}')
```

Question : Listez les bibliothèques pré-installées et leurs versions. Lesquelles auraient nécessité une installation manuelle sur une machine locale ?

Exercice 1.3 — Installer une bibliothèque externe

Installez une bibliothèque qui n'est PAS pré-installée :

Cellule Colab

```
!pip install transformers -q

from transformers import pipeline

# Test : analyse de sentiment
analyseur = pipeline('sentiment-analysis')
resultat = analyseur('I love artificial intelligence!')
```

```
print(resultat)
```

Question : Quel résultat obtenez-vous ? Que fait la fonction pipeline() de HuggingFace ? Pourquoi cette bibliothèque n'est-elle pas pré-installée ?

Exercice 1.4 — Manipulation de données avec Pandas

Cellule Colab

```
# Créer un DataFrame
data = {
    'Pays': ['Togo', 'Burkina Faso', 'Sénégal', 'Côte d\'Ivoire', 'Ghana'],
    'Population_M': [8.8, 22.1, 17.7, 28.2, 33.5],
    'PIB_Mds_USD': [8.1, 19.7, 27.6, 70.0, 77.6],
    'Pénétration_Internet_%': [23, 22, 58, 36, 53]
}

df = pd.DataFrame(data)
print(df)
print(f'\nMoyenne PIB : {df.PIB_Mds_USD.mean():.1f} Mds $')
print(f'Pays plus connecté : {df.loc[df["Pénétration_Internet_%"].idxmax(), "Pays"]}')
```

Question : Quel pays a la meilleure pénétration Internet ? Quel est le PIB moyen ? Ajoutez une colonne PIB_par_habitant et identifiez le pays le plus riche par habitant.

Exercice 1.5 — Visualisation avec Matplotlib

Cellule Colab

```
plt.figure(figsize=(10, 5))

# Graphique 1 : barres
plt.subplot(1, 2, 1)
plt.bar(df['Pays'], df['PIB_Mds_USD'],
color=['#028090','#E8A317','#028090','#E8A317','#028090'])
plt.title('PIB par pays (Mds USD)')
plt.xticks(rotation=45)

# Graphique 2 : courbe
plt.subplot(1, 2, 2)
x = np.linspace(0, 4*np.pi, 200)
plt.plot(x, np.sin(x), label='sin(x)', color='#028090', linewidth=2)
plt.plot(x, np.cos(x), label='cos(x)', color='#E8A317', linewidth=2)
plt.title('Fonctions trigonométriques')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Capture d'écran des deux graphiques.

Partie 2 — Frameworks ML en pratique

Exercice 2.1 — Scikit-learn : ML classique

Réalisez une classification complète sur le dataset Iris :

Cellule Colab

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Chargement
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

# Entraînement
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Évaluation
preds = model.predict(X_test)
print(f'Précision : {accuracy_score(y_test, preds):.2%}')
print(classification_report(y_test, preds, target_names=iris.target_names))
```

Bonus : affichez la matrice de confusion avec plt.imshow() et les importances des features avec model.feature_importances_.

Exercice 2.2 — PyTorch : premier réseau de neurones

Construisez un réseau de neurones simple pour classifier les chiffres MNIST :

Cellule Colab

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Vérification GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Utilisation de : {device}')

# Données MNIST
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,),
(0.5,))])
train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_data = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64)

print(f'Images entraînement : {len(train_data)}')
```

```
print(f'Images test : {len(test_data)}')
```

Cellule suivante — Définir le modèle :

Cellule Colab

```
class ReseauSimple(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.couches = nn.Sequential(
            nn.Linear(28*28, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.couches(x)

model = ReseauSimple().to(device)
print(model)
print(f'Paramètres : {sum(p.numel() for p in model.parameters()):,}')
```

Cellule suivante — Entraîner :

Cellule Colab

```
critere = nn.CrossEntropyLoss()
optimiseur = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimiseur.zero_grad()
        sorties = model(images)
        loss = critere(sorties, labels)
        loss.backward()
        optimiseur.step()
        total_loss += loss.item()
    print(f'Epoch {epoch+1}/5 — Loss : {total_loss/len(train_loader):.4f}')
```

Cellule suivante — Évaluer :

Cellule Colab

```
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        sorties = model(images)
        _, predits = torch.max(sorties, 1)
        total += labels.size(0)
        correct += (predits == labels).sum().item()

print(f'Précision sur le test : {100*correct/total:.2f}%')
```

Questions :

Q1 : Combien de paramètres a votre modèle ? Que fait nn.Flatten() ?

Q2 : Quelle précision obtenez-vous ? L'entraînement a-t-il utilisé le GPU (vérifiez avec device) ?

Q3 : Que fait loss.backward() ? Et optimiseur.step() ? Expliquez en une phrase chacun.

Exercice 2.3 — TensorFlow/Keras : même tâche

Refaites la même classification MNIST mais avec Keras :

Cellule Colab

```
import tensorflow as tf
from tensorflow import keras

# Données
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 28*28).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28*28).astype('float32') / 255.0

# Modèle
model_keras = keras.Sequential([
    keras.layers.Dense(256, activation='relu', input_shape=(784,)),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model_keras.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model_keras.summary()

# Entraînement
history = model_keras.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1,
verbose=1)

# Évaluation
test_loss, test_acc = model_keras.evaluate(x_test, y_test)
print(f'\nPrécision test Keras : {test_acc:.2%}')
```

Questions :

Q1 : Comparez le nombre de lignes de code entre PyTorch et Keras. Lequel est plus concis ?

Q2 : Les précisions sont-elles similaires ? Pourquoi ?

Q3 : Quelle différence voyez-vous dans la boucle d'entraînement (PyTorch: boucle manuelle vs Keras: model.fit) ?

Partie 3 — Questions théoriques

Q1 : Pourquoi Python domine en IA à plus de 90% ? Citez au moins 3 raisons et donnez un exemple pour chacune.

Q2 : Comparez VS Code, Jupyter Notebook et Google Colab. Pour chaque outil, donnez un cas d'usage idéal.

Q3 : Comparez PyTorch et TensorFlow/Keras. Lequel recommanderiez-vous à un débutant ? À un chercheur ? À une entreprise qui déploie en production ?

Q4 : Qu'est-ce que Scikit-learn ? Pourquoi dit-on qu'il couvre 80% des cas en entreprise ? Donnez un exemple de problème que Scikit-learn résout et un que PyTorch résout mieux.

Q5 : Citez 3 limites de Google Colab par rapport à un serveur GPU dédié. Dans quel contexte Colab est-il malgré tout le meilleur choix ?

Q6 : Quel est le rôle de C++ dans l'écosystème IA même si tout le monde code en Python ?

Q7 : Qu'est-ce que HuggingFace Transformers ? Pourquoi est-il devenu incontournable en NLP ?

Partie 4 — Mini-projet : comparaison de modèles

Créez un notebook Colab complet qui compare 3 approches sur le dataset Digits (sklearn) :

Approche 1 : Scikit-learn — RandomForestClassifier

Approche 2 : PyTorch — Réseau de neurones simple (2 couches cachées)

Approche 3 : Keras — Modèle Sequential équivalent

Le notebook doit contenir : le chargement des données, l'entraînement des 3 modèles, la comparaison des précisions dans un tableau récapitulatif, un graphique comparant les précisions, et une conclusion justifiée sur le meilleur choix pour ce dataset.